

Defend-The-Core

Infrastructure PME critique sécurisée

Annexe 4 — Administration sécurisée & Bastion NixOS

Le Privileged Access Workstation (PAW) : point unique d'administration immuable

Immuabilité déclarative · ProxyJump · FIDO2 · Défense en profondeur

« La star du projet : si un attaquant modifie `/etc/ssh/sshd_config`, NixOS l'écrasera au prochain déploiement. »

Table des matières

1. Le concept de PAW (Privileged Access Workstation)	3
1.1. Chaîne d'administration relayée	3
2. Pourquoi NixOS ?	3
2.1. L'argument central : l'immuabilité anti-persistance	4
2.2. Comparaison avec d'autres systèmes	4
3. ProxyJump (administration relayée)	4
3.1. Configuration client (ssh_config)	5
3.2. known_hosts et protection anti-MITM	5
3.3. Clés dédiées par usage	5
4. configuration.nix — analyse détaillée	6
4.1. Serveur SSH durci	6
4.2. Pare-feu local (iptables)	7
4.3. Paramètres noyau (sysctl) durcis	7
4.4. Immuabilité et anti-persistance	8
4.5. Auditd — surveillance de l'intégrité	8
4.6. fail2ban — protection anti-brute-force	9
4.7. Empreinte minimale des services	9
4.8. Logs remontés vers Wazuh	9
5. Clé FIDO2 (facteur matériel)	10
5.1. Résistance au phishing et à l'exfiltration	10
6. Checklist de durcissement	11
7. L'argument « Waouh » pour l'entretien	11

1. Le concept de PAW (Privileged Access Workstation)

Un **Privileged Access Workstation (PAW)**, ou bastion d'administration, est une machine dédiée et durcie dont l'unique fonction est de servir de **point unique d'administration** vers les serveurs critiques. L'administrateur ne se connecte *jamais* directement à un serveur de production depuis son poste de travail ordinaire, fût-il de confiance : tout le trafic d'administration transite par le bastion, qui se trouve dans une zone réseau isolée (ici le VLAN 99 — Admin/SIEM).

La rationale est double. D'abord, le poste de l'administrateur reste une surface d'attaque : navigateur, messagerie, documents en pièce jointe. Un phishing réussi sur ce poste exposerait immédiatement les identifiants SSH vers les serveurs si la connexion était directe. Ensuite, centraliser l'administration sur un hôte unique permet de **tracer, durcir et verrouiller** l'ensemble des sessions privilégiées au même endroit : pare-feu local, journalisation, authentification forte, immuabilité.

1.1. Chaîne d'administration relayée

Le flux d'administration respecte une chaîne stricte à trois sauts. Chaque maillon impose une authentification distincte et un contrôle réseau explicite :

```
Poste admin (bureautique, VLAN 10)
| (1) SSH + clé FID02 (ed25519-sk)
▼
Bastion NixOS (10.10.99.20, VLAN 99)
| (2) ProxyJump ssh -J (clé ed25519 dédiée)
▼
Serveurs cibles (10.10.30.0/24, VLAN 30)
```

Figure 1 — Chaîne d'administration relayée. Le poste admin n'a aucune visibilité du VLAN 30 ; seul le bastion (10.10.99.20) peut joindre le port 22 des serveurs, et uniquement parce qu'OPNsense l'y autorise explicitement.

OPNsense applique une règle **default-deny** sur le VLAN 30 : la seule exception autorisant le port 22 a pour source unique le bastion (10.10.99.20). Toute tentative SSH directe depuis le VLAN 10, ou même depuis un autre hôte du VLAN 99, est rejetée et journalisée. L'administration devient donc **impossible hors bastion** — c'est le contrôle d'accès le plus puissant du projet, car il ne dépend pas d'un mot de passe mais de la topologie réseau elle-même.

2. Pourquoi NixOS ?

Le choix de NixOS comme système d'exploitation du bastion est la décision la plus structurante du projet. NixOS repose sur deux principes qui le distinguent radicalement des distributions Linux classiques : la **configuration déclarative** et l'**immuabilité de l'état système**.

L'intégralité de la configuration — paquets installés, services activés, options SSH, règles de pare-feu, paramètres sysctl, utilisateurs — est décrite dans un fichier unique, `configuration.nix`. Appliquer la configuration se fait par `nixos-rebuild switch`, qui génère un nouveau *profil* système atomique. Les profils précédents restent disponibles, ce qui autorise un **rollback instantané** (`nixos-rebuild switch --rollback`) en cas de régression.

2.1. L'argument central : l'immuabilité anti-persistance

Sur une distribution classique (Ubuntu, Debian), un attaquant qui obtient les droits root peut éditer `/etc/ssh/sshd_config`, `/etc/pam.d/sshd` ou déposer un binaire dans `/usr/local/bin` ; ces modifications **persistent** à travers les redémarrages. La backdoor est installée durablement.

Sur NixOS, la situation est fondamentalement différente. Les fichiers de configuration système ne sont pas édités directement : ils sont générés depuis `configuration.nix` à chaque déploiement et placés dans le **Nix store** en lecture seule (`/nix/store`, monté en `readOnly`). Toute modification manuelle d'un fichier géré par NixOS est **écrasée** au prochain `nixos-rebuild switch`. Le Nix store étant immuable, il est impossible d'y remplacer un binaire en place.

Argument clé. Si un attaquant modifie `/etc/ssh/sshd_config` pour réactiver l'authentification par mot de passe ou ajouter une clé, NixOS l'écrasera au prochain déploiement. L'immuabilité empêche la persistance : la backdoor ne survit pas à un `nixos-rebuild switch`.

2.2. Comparaison avec d'autres systèmes

Le tableau suivant compare NixOS aux distributions classiques sur les critères qui comptent pour un bastion d'administration. L'objectif n'est pas de dénigrer Ubuntu ou Debian — excellents pour les serveurs métier durcis via CIS Benchmark — mais de mettre en évidence la propriété unique de NixOS : la **non-persistance** des modifications non déclarées.

Tableau 1 — NixOS face aux distributions classiques

Critère	NixOS	Ubuntu	Debian
Immuabilité de la configuration	Totale : config régénérée depuis <code>configuration.nix</code>	Partielle : fichiers éditables manuellement	Partielle : fichiers éditables manuellement
Reproductibilité	Exacte : même config → même état	Manuelle : dépend de l'historique apt	Manuelle : dépend de l'historique apt
Rollback	Instantané (<code>--rollback</code>), profil atomique	Non natif, à la main	Non natif, reconstruction manuelle
Versionning Git	Natif : <code>configuration.nix</code> versionnée	Indirect : ansible/puppet à ajouter	Indirect : outil de config management
Persistance d'une backdoor	Impossible : écrasée au rebuild	Possible : modification manuelle persiste	Possible : modification manuelle persiste
Surface de déploiement	Un fichier, déclaratif, audité	Scripts + état divergent possible	Scripts + état divergent possible

Cette propriété d'anti-persistance est ce qui justifie NixOS *spécifiquement* pour le bastion, là où Ubuntu et Debian suffisent pour les serveurs métier (VLAN 30) dont la configuration est durcie mais reste ponctuellement éditale.

3. ProxyJump (administration relayée)

Le **jump host** est un patron classique d'administration sécurisée : au lieu d'exposer le SSH des serveurs cibles à un large réseau, on force tout le trafic à transiter par un hôte intermédiaire verrouillé. OpenSSH formalise ce mécanisme depuis sa version 7.3 via l'option **ProxyJump** (-J), qui établit automatiquement le tunnel SSH vers la cible à travers le bastion.

Concrètement, l'administrateur tape une seule commande : `ssh -J bastion serveur`. OpenSSH ouvre d'abord une connexion SSH vers **bastion**, puis depuis le bastion ouvre une seconde connexion TCP vers **serveur** sur son port 22, et encapsule la session finale dans ce tunnel. La clé privée du serveur cible ne quitte jamais le poste admin — elle est utilisée localement au travers du canal relayé.

3.1. Configuration client (ssh_config)

Plutôt que de saisir -J à chaque fois, la chaîne est déclarée dans `~/.ssh/config` du poste admin. Deux hôtes sont définis : le bastion (authentification FIDO2) et les cibles (relayées via le bastion, clé ed25519 dédiée).

```
# ~/.ssh/config sur le poste admin

Host bastion
  HostName 10.10.99.20
  User admin
  IdentityFile ~/.ssh/id_ed25519_sk # clé FIDO2 (matérielle)
  PasswordAuthentication no
  PubkeyAuthentication yes

Host srv-*
  ProxyJump bastion # relais obligatoire
  User deploy
  IdentityFile ~/.ssh/id_ed25519_srv # clé dédiée serveurs
  IdentitiesOnly yes
  PasswordAuthentication no
```

Extrait 1 — ssh_config : chaîne ProxyJump et clés dédiées par usage.

3.2. known_hosts et protection anti-MITM

Pour empêcher une attaque de type **man-in-the-middle** sur le premier saut, l'empreinte de la clé publique du bastion est pré-positionnée dans `~/.ssh/known_hosts` via le dépôt Git (procédure de premier déploiement). OpenSSH refuse alors toute connexion si la clé présentée diffère de l'empreinte attendue. Les empreintes des serveurs du VLAN 30 sont également vérifiées au second saut, ce qui verrouille les deux maillons de la chaîne.

3.3. Clés dédiées par usage

Tableau 2 — Clés SSH dédiées par usage

Usage	Type de clé	Facteur	Justification
Accès au bastion	ed25519-sk	Matériel (FIDO2)	Anti-clonage : la clé ne sort jamais du token
Accès aux serveurs	ed25519	Logiciel	Légère, usage relayé, stockée sur le poste admin

Root (serveurs)	Interdit	—	Pas de PermitRootLogin, sudo journalisé uniquement
-----------------	----------	---	--

L'OPNsense autorise uniquement le bastion (10.10.99.20) à joindre le port 22 du VLAN 30. Tout autre source est rejetée et journalisée : la topologie rend l'administration directe impossible, indépendamment des identifiants.

4. configuration.nix — analyse détaillée

Cette section décompose la configuration du bastion, bloc par bloc. Le fichier complet est versionné dans le dépôt (bastion/configuration.nix) ; les extraits ci-dessous en montrent les parties significatives avec leur rationale de sécurité. Chaque directive répond à une recommandation précise du guide de durcissement ANSSI / CIS.

4.1. Serveur SSH durci

Le serveur SSH du bastion est l'unique point d'entrée d'administration. Il est verrouillé sur l'essentiel : pas de root, pas de mot de passe, authentification par clé FIDO2 uniquement, algorithmes cryptographiques modernes et limités.

```
services.openssh = {
  enable = true;
  settings = {
    PermitRootLogin = "no"; # root interdit
    PasswordAuthentication = false; # aucune auth. par mot de passe
    KbdInteractiveAuthentication = false;
    PubkeyAuthentication = true;
    MaxAuthTries = 3; # 3 essais, puis échec
    LoginGraceTime = 30; # 30 s pour s'authentifier
    AllowUsers = [ "admin" ]; # liste blanche
  };
  banner = /etc/ssh/banner; # avertissement légal
};
```

Extrait 2 — services.openssh : authentification par clé uniquement, root interdit.

La directive `PermitRootLogin = "no"` supprime la possibilité même d'une connexion root directe — l'administrateur se connecte en compte non privilégié, puis élève via `sudo` (journalisé). `PasswordAuthentication = false` neutralise entièrement le bruteforce de mots de passe, et `MaxAuthTries = 3` limite le nombre de tentatives par session avant déconnexion.

Bannière légale. Un fichier `/etc/ssh/banner` affiche un avertissement : « Accès réservé à l'administration autorisée. Toute connexion est journalisée et surveillée. » Outre l'effet dissuasif, cette bannière est requise pour la recevabilité juridique des journaux en cas de poursuite.

Algorithmes cryptographiques retenus

Tableau 3 — Suite cryptographique durcie (KexAlgorithms, Ciphers, MACs)

Famille	Algorithmes autorisés	Algorithmes écartés	Raison
---------	-----------------------	---------------------	--------

Clé serveur	rsa-sha2-512, rsa-sha2-256, ssh-ed25519	ssh-rsa (SHA-1), DSA	Faiblesses cryptographiques (SHA-1)
Échange de clé	curve25519-sha256, ecdh-sha2-nistp521	diffie-hellman-group1, group14-sha1	DH faibles, obsolètes
Chiffrement	chacha20-poly1305, aes256-gcm	3des-cbc, aes128-cbc, arcfour	Modes obsolètes, faibles
Intégrité (MAC)	hmac-sha2-512-etm, hmac-sha2-256-etm	hmac-sha1, hmac-md5	Fonctions de hachage cassées

Restreindre la liste des algorithmes à un jeu moderne et homogène réduit la surface d'attaque (pas de négociation vers un algorithme faible) et facilite l'audit : un algorithme nouveau ou inattendu apparaissant dans les logs trahit immédiatement une anomalie.

4.2. Pare-feu local (iptables)

Outre le filtrage d'OPNsense à l'échelle du réseau, le bastion applique son propre pare-feu local en **default-deny**. Cette défense en profondeur signifie que même une erreur de configuration d'OPNsense ne suffirait pas à exposer le bastion : seuls les flux explicitement listés ci-dessous sont acceptés.

```
networking.firewall = {
  enable = true;
  allowedTCPPorts = [ 22 ]; # SSH entrant
  interfaces.eth0.allowedTCPPorts = [ 22 ]; # VLAN 99 uniquement
  extraCommands = ''
    iptables -A INPUT -s 10.10.99.0/24 -p tcp --dport 22 -j ACCEPT
    iptables -A INPUT -s 10.10.10.0/24 -j DROP # blocage VLAN 10
    iptables -A INPUT -s 10.10.20.0/24 -j DROP # blocage VLAN 20
    iptables -A OUTPUT -d 10.10.30.0/24 -p tcp --dport 22 -j ACCEPT
    iptables -A OUTPUT -j DROP # reste refusé
  '';
};
```

Extrait 3 — Pare-feu local : SSH entrant depuis le VLAN 99 uniquement, SSH sortant vers le VLAN 30 uniquement, blocage des VLAN 10 et 20.

Tableau 4 — Politique de pare-feu local du bastion

Sens	Source	Destination	Port	Action	Justification
Entrant	VLAN 99	Bastion	22	Allow	Admin via FIDO2 uniquement
Entrant	VLAN 10/20	Bastion	*	Drop	Bureautique/DMZ isolées
Sortant	Bastion	VLAN 30	22	Allow	ProxyJump vers serveurs
Sortant	Bastion	VLAN 10/20	*	Drop	Pas de retour vers zones moins fiables
Sortant	Bastion	Wazuh (99.10)	514	Allow	Forward des logs

4.3. Paramètres noyau (sysctl) durcis

Le durcissement noyau complète la posture défensive. Ces paramètres rendent plus difficiles l'exploitation de vulnérabilités mémoire, l'usurpation réseau et la reconnaissance interne. Ils sont appliqués via

`boot.kernel.sysctl` et donc, comme toute la configuration, régénérés à chaque déploiement.

Tableau 5 — Paramètres `sysctl` durcis du bastion

Paramètre	Valeur	Objectif
<code>kernel.randomize_va_space</code>	2	ASLR complet (adressage aléatoire) — complique les exploits mémoire
<code>kernel.yama.ptrace_scope</code>	2	Restreint <code>ptrace</code> au root ; empêche l'inspection d'un process par un autre
<code>net.ipv4.conf.all.rp_filter</code>	1	Anti-spoofing : rejette les paquets à source incohérente avec l'interface
<code>net.ipv4.tcp_syncookies</code>	1	Résistance au SYN flood (épuisement de la table des connexions)
<code>net.ipv4.conf.all.log_martians</code>	1	Journalise les paquets impossibles/illégaux (détection d'attaque)
<code>net.ipv4.icmp_echo_ignore_broadcasts</code>	1	Anti-smurf : ignore les echo broadcasts
<code>net.ipv4.ip_forward</code>	0	Pas de routage : le bastion n'est pas un routeur
<code>fs.suid_dumpable</code>	0	Pas de core dump pour les binaires <code>setuid</code> (fuite d'infos)

4.4. Immuabilité et anti-persistance

L'immuabilité est renforcée au niveau du boot. Le Nix store est monté en lecture seule et `/tmp` est placé sur un `tmpfs` (en RAM), ce qui garantit qu'aucun artefact écrit à chaud ne survive à un redémarrage.

```
boot.readOnlyNixStore = true; # /nix/store immuable
boot.tmpOnTmpfs = true; # /tmp en RAM, effacé au reboot
boot.cleanTmpDir = true; # nettoyage /tmp au démarrage

# La configuration est versionnée Git : /etc/nixos est un dépôt
# clone du repository du projet. Tout changement est un commit.
```

Extrait 4 — Immuabilité du Nix store et de `/tmp`.

La combinaison `readOnlyNixStore` + `tmpOnTmpfs` crée une propriété forte : un attaquant qui réussirait à déposer un binaire malveillant dans `/tmp` le verrait disparaître au prochain redémarrage, et ne pourrait pas le recopier dans le Nix store (lecture seule). La configuration étant versionnée Git, toute modification légitime passe par un commit auditable et une revue de pull request.

4.5. Auditd — surveillance de l'intégrité

Le sous-système `auditd` journalise tout accès ou modification des fichiers sensibles, ainsi que l'exécution de commandes privilégiées. Les événements sont remontés en temps réel vers Wazuh (section 4.8).

```
security.audit.enable = true;
security.audit.rules = ''
  -w /etc/nixos/ -p wa -k nixos_config # config NixOS
  -w /etc/sudoers -p wa -k sudoers # privilèges sudo
  -w /etc/passwd -p wa -k identity # comptes
  -w /etc/shadow -p wa -k identity # mots de passe
  -w /var/log/auth.log -p wa -k authlog # logs d'auth.
  -a always,exit -F arch=b64 -S execve \
    -F path=/usr/bin/sudo -k privileged
'';
```


Toute écriture (-p w) ou modification d'attribut (-p a) sur /etc/nixos, /etc/sudoers ou /etc/passwd génère un événement horodaté et taggé. Wazuh corrèle ces événements avec les connexions SSH pour détecter, par exemple, une modification de configuration juste après une connexion inhabituelle.

4.6. fail2ban — protection anti-bruteforce

Bien que l'authentification par mot de passe soit désactivée, fail2ban reste pertinent : il détecte les tentatives répétées d'authentification échouée (clés refusées, erreurs de protocole) et bannit l'IP source. La stratégie est un **ban incrémental** d'une heure après 3 échecs.

```
services.fail2ban = {
  enable = true;
  jails.sshd = ''
    enabled = true
    maxretry = 3 # 3 essais avant ban
    findtime = 10m # fenêtre de 10 min
    bantime = 1h # ban 1 heure
    bantime.increment = true # ban plus long aux récidives
  '';
};
```

Extrait 6 — fail2ban : 3 essais, ban d'une heure, incrémental.

4.7. Empreinte minimale des services

Un bastion ne fait qu'une chose : relayer l'administration. Aucun service inutile n'est exposé. Pas d'environnement graphique, pas de serveur Web, pas d'imprimante, pas de Samba. La liste des paquets est réduite au strict nécessaire et déclarée explicitement. La synchronisation temporelle (NTP) est maintenue pour permettre la corrélation temporelle des événements dans le SIEM.

```
environment.systemPackages = with pkgs; [
  openssh git vim tcpdump htop nix-prefetch-scripts
];
services.xserver.enable = false; # pas de X11
services.printing.enable = false; # pas d'impression
services.avahi.enable = false; # pas de mDNS

services.ntp.enable = true; # heure précise pour SIEM
services.chrony.enable = true; # source de temps robuste
```

Extrait 7 — Empreinte minimale : pas de X11, paquets restreints, NTP pour le SIEM.

La **synchronisation NTP** est essentielle : sans horloges cohérentes, les événements remontés par auditd, SSH et fail2ban ne peuvent être corrélés correctement dans Wazuh. Une dérive d'horloge de quelques secondes suffit à masquer la séquence réelle d'une attaque.

4.8. Logs remontés vers Wazuh

Tous les journaux du bastion — authentifications SSH, règles auditd, bannissements fail2ban, déploiements nixos-rebuild — sont transférés vers Wazuh (10.10.99.10) via rsyslog sur le port 514. Le bastion ne conserve qu'une copie locale ; la source de vérité pour la corrélation est le SIEM.

```

services.rsyslogd.enable = true;
services.rsyslogd.defaultConfig = ''
    *.* @10.10.99.10:514 # forward TCP vers Wazuh
    auth,authpriv.* /var/log/auth.log # copie locale
'';

```

Extrait 8 — rsyslog : forward des logs vers Wazuh (10.10.99.10:514).

Le forward en TCP (et non UDP) garantit la délivrance : en cas d'indisponibilité temporaire de Wazuh, rsyslog bufferise et retransmet. Wazuh corrèle alors les événements du bastion avec ceux des autres hôtes (OPNsense, serveurs du VLAN 30) pour reconstituer la chronologie complète d'une session d'administration.

5. Clé FIDO2 (facteur matériel)

L'accès au bastion exige une clé SSH de type **ed25519-sk** (*sk* pour *security key*). Cette clé est stockée dans un token matériel FIDO2 — une clé USB physique de type YubiKey — et non dans un fichier sur disque. Le suffixe **-sk** indique à OpenSSH que la partie privée de la clé réside sur le token et qu'une opération de signature doit déléguer au matériel.

La génération se fait en une commande, après avoir inséré le token :

```

$ ssh-keygen -t ed25519-sk -C "admin@bastion"
# Le token demande un toucher physique + un PIN FIDO2
# → ~/.ssh/id_ed25519_sk (handle public, sans secret)
# → ~/.ssh/id_ed25519_sk.pub (à déposer dans authorized_keys du bastion)

```

Extrait 9 — Génération d'une clé SSH FIDO2 (ed25519-sk).

5.1. Résistance au phishing et à l'exfiltration

Contrairement à une clé logicielle classique (fichier `id_ed25519`), une clé FIDO2 ne peut pas être **clonée** ni **copiée** à distance : la partie privée ne quitte jamais le token. Même si le poste administrateur est entièrement compromis par un malware, l'attaquant ne peut pas voler la clé — il peut au pire *utiliser* le token pendant qu'il est physiquement inséré, ce qui suppose déjà un compromis runtime actif.

Tableau 6 — Clé logicielle vs clé FIDO2

Critère	Clé logicielle (ed25519)	Clé matérielle (ed25519-sk)
Stockage du secret privé	Fichier sur disque (~/.ssh/id_ed25519)	Token FIDO2, non exportable
Résistance au phishing	Faible : la clé peut être volée par malware	Forte : secret non clonable
Facteur utilisateur	Aucun (passphrase optionnelle)	Toucher + PIN FIDO2
Compromission du poste	Clé exfiltrable à distance	Clé non exfiltrable
Perte du support	—	Clé inservible sans le token

La clé FIDO2 apporte donc deux garanties décisives pour un bastion : **anti-clonage** (le secret reste matériel) et **présence physique** (chaque signature demande un toucher, ce qui empêche l'usage automatisé à l'insu de l'administrateur). C'est l'incarnation du principe d'authentification forte recommandé par l'ANSSI pour les accès privilégiés.

6. Checklist de durcissement

Le tableau suivant récapitule l'ensemble des mesures appliquées au bastion, regroupées par domaine. Il sert de référence d'audit et de support de présentation en entretien : chaque ligne répond à une question concrète « comment ? » et « pourquoi ? ».

Tableau 7 — Checklist de durcissement du bastion NixOS

Domaine	Mesure	Implémentation	Justification
Authentification	Root interdit	PermitRootLogin = "no"	Pas de connexion root directe
Authentification	Mot de passe interdit	PasswordAuthentication = false	Neutralise le bruteforce SSH
Authentification	Clé FIDO2 obligatoire, 3 essais max	ed25519-sk, AllowUsers admin, MaxAuthTries = 3	Facteur matériel anti-clonage
Cryptographie	Algorithmes modernes	chacha20-poly1305, ed25519, SHA-2 + bannière légale	Écarte SHA-1, DSA, 3DES, CBC
Immuabilité	Nix store lecture seule	boot.readOnlyNixStore = true	Binaires système non remplaçables
Immuabilité	/tmp en RAM	boot.tmpOnTmpfs = true	Anti-persistance des artefacts
Immuabilité	Config versionnée Git + rollback	/etc/nixos = dépôt Git, nixos-rebuild --rollback	Audit, revue, retour arrière instantané
Réseau	Pare-feu local default-deny	networking.firewall + iptables	Défense en profondeur
Réseau	SSH entrant VLAN 99, sortant VLAN 30	iptables -s/-d sur sous-réseaux	Zone admin isolée, ProxyJump ciblé
Réseau	Blocage VLAN 10/20	iptables -j DROP	Isolation zones moins fiables
Audit	Surveillance fichiers clés + sudo	auditd -w /etc/nixos, sudoers, passwd, sudo	Détection de modification et d'élévation
Audit	Logs vers SIEM	rsyslog @@10.10.99.10:514	Corrélation centralisée
sysctl	ASLR complet	kernel.randomize_va_space = 2	Complique l'exploitation mémoire
sysctl	ptrace restreint	kernel.yama.ptrace_scope = 2	Anti-inspection de processus
sysctl	Anti-spoofing / SYN flood / martians	rp_filter=1, tcp_syncookies=1, log_martians=1	Robustesse et détection réseau
Services	Pas d'environnement graphique	services.xserver.enable = false	Réduction de la surface
Services	NTP activé	services.chrony.enable = true	Corrélation temporelle SIEM
ProxyJump	Relais obligatoire + anti-MITM	ssh -J bastion, known_hosts verrouillé	Aucune route directe vers VLAN 30

7. L'argument « Waouh » pour l'entretien

En entretien, le bastion NixOS est l'élément qui distingue ce projet d'un durcissement Linux classique. La force de la démonstration tient en quatre piliers qui, combinés, réalisent une posture d'administration privilégiée rarement atteinte même dans des SI plus matures.

Tableau 8 — Les quatre piliers du bastion

Pilier	Mise en œuvre	Bénéfice
1. Authentification forte	Clé FIDO2 (ed25519-sk), root interdit, 3 essais	Anti-clonage, anti-bruteforce, facteur matériel
2. Immuabilité	NixOS déclaratif, Nix store RO, /tmp en RAM, Git	Anti-persistance : la backdoor ne survit pas
3. Défense en profondeur	Pare-feu OPNsense + pare-feu local + sysctl + fail2ban	Plusieurs couches indépendantes à franchir
4. Administration tracée	ProxyJump, auditd, logs forwardés vers Wazuh, NTP	Toute session est journalisée et corrélable

Le pitch en une phrase

« Si un attaquant modifie `/etc/ssh/sshd_config` pour ajouter un mot de passe, NixOS l'écrasera au prochain déploiement. L'immuabilité empêche la persistance. »

Cette phrase résume à elle seule la valeur ajoutée du bastion : là où un durcissement classique rend l'attaque *difficile*, l'immuabilité déclarative de NixOS la rend *non durable*. Combinée à l'authentification FIDO2 (anti-clonage), au ProxyJump (pas de route directe vers les serveurs) et à la journalisation centralisée (toute session est corrélée dans Wazuh), elle réalise une posture d'administration privilégiée alignée sur les recommandations ANSSI — et argumentable en entretien avec une démonstration reproductible.

Cette annexe est le quatrième volet de la documentation *Defend-The-Core*. Les annexes 1 (Réseau & OPNsense), 2 (Wazuh / SIEM) et 3 (Durcissement des serveurs) détaillent les autres domaines de la posture de sécurité.